Homework 1

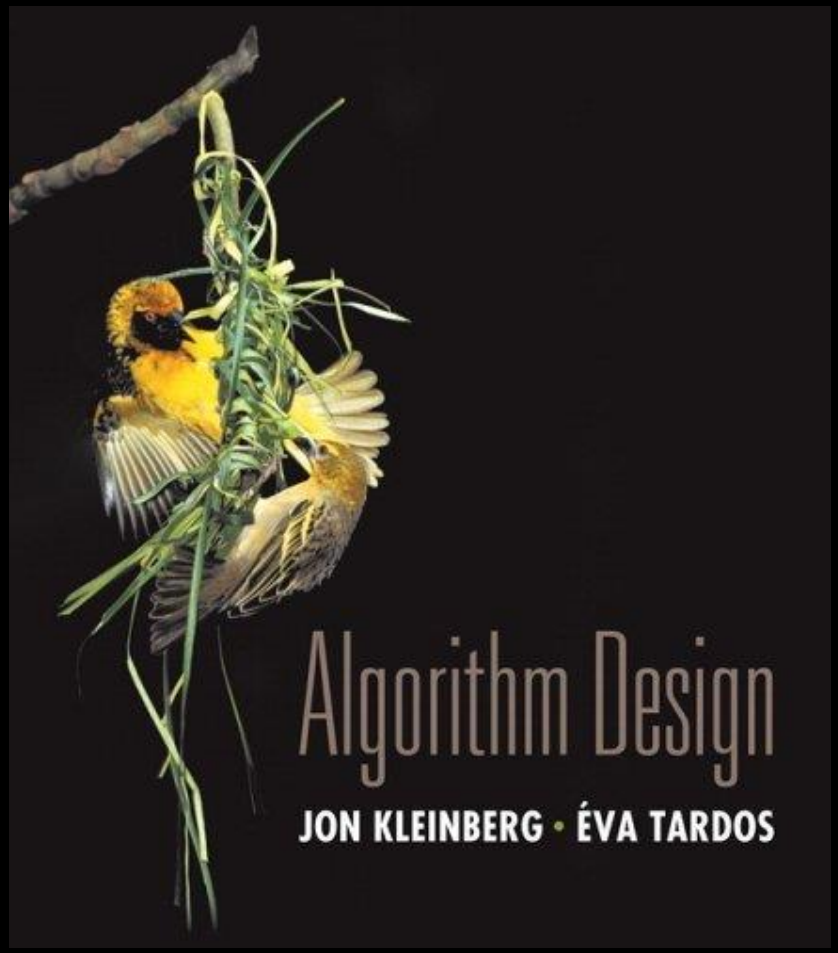
Homework 1

- Released today in Blackboard
- Due: 11:59pm, Mar. 18

- No late homework will be accepted!

Submission

- Register at gradescope.com using your university email
  - Make sure that your FULL NAME is your Chinese name and your STUDENT ID is correct
- Save your solutions in a PDF or image file
- Upload your file to Gradescope and match your solution to each problem



Chapter 4

Greedy Algorithms

Greedy Algorithms

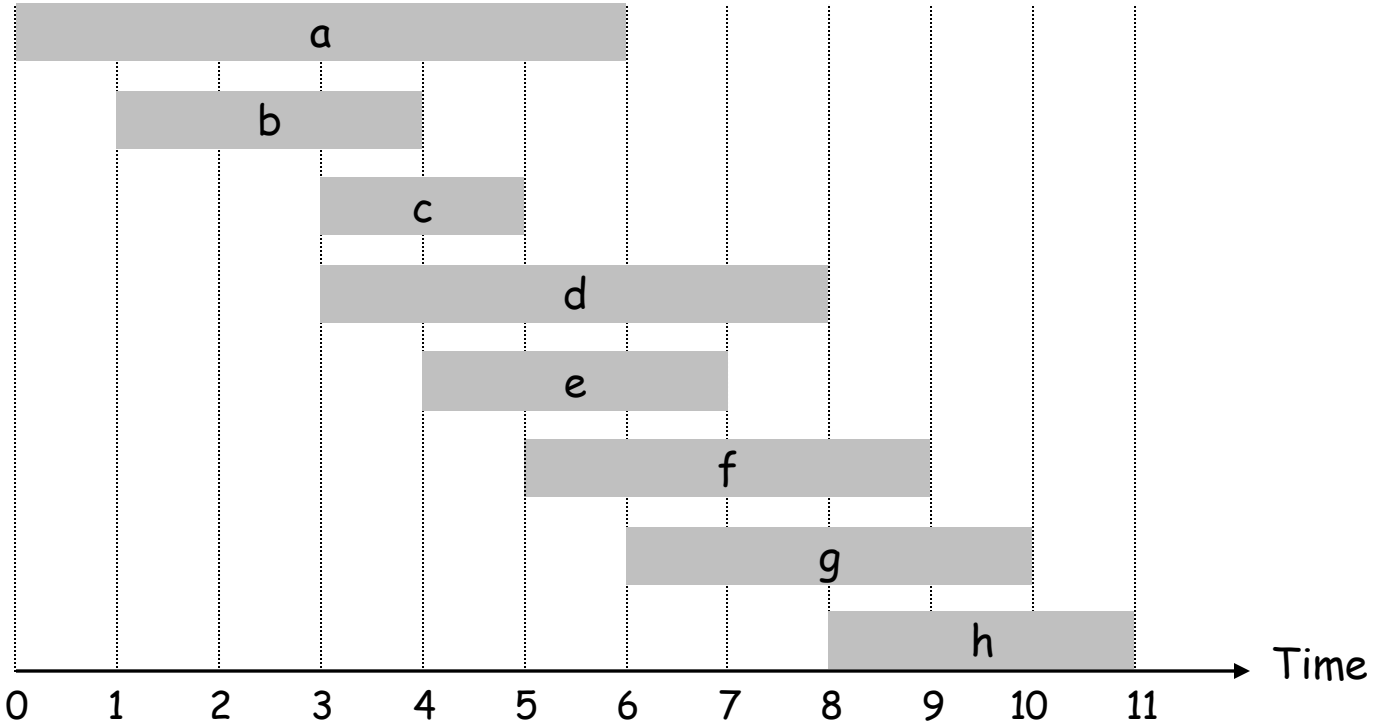
A greedy algorithm is an algorithm that makes the locally optimal choice at each step.

4.1 Interval Scheduling

Interval Scheduling

Interval scheduling.

- Job j starts at s_j and finishes at f_j.
- Two jobs compatible if they don't overlap.
- Goal: find maximum subset of mutually compatible jobs.



Interval Scheduling: Greedy Algorithms

Greedy template. Consider jobs in some order. Take each job provided it's compatible with the ones already taken.

- [Earliest start time] Consider jobs in ascending order of start time s_j.
- [Earliest finish time] Consider jobs in ascending order of finish time f_j.
- [Shortest interval] Consider jobs in ascending order of interval length $f_j - s_j$.
- [Fewest conflicts] For each job, count the number of conflicting jobs c_j. Schedule in ascending order of conflicts c_j.

Interval Scheduling: Greedy Algorithms

Greedy template. Consider jobs in some order. Take each job provided it's compatible with the ones already taken.

earliest start time?

shortest interval?

fewest conflicts?

Interval Scheduling: Greedy Algorithm

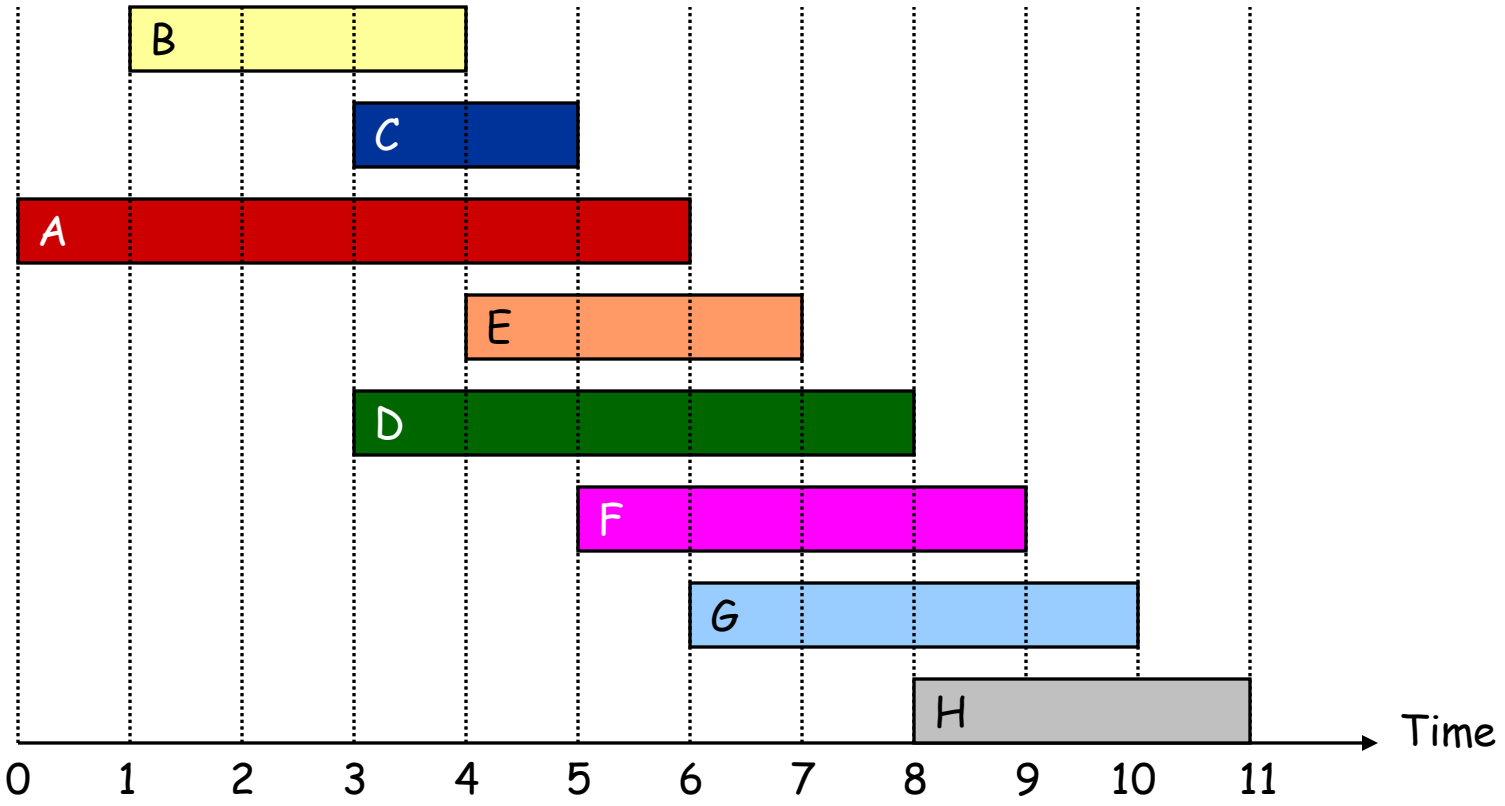
Greedy algorithm. Consider jobs in increasing order of finish time. Take each job provided it's compatible with the ones already taken.

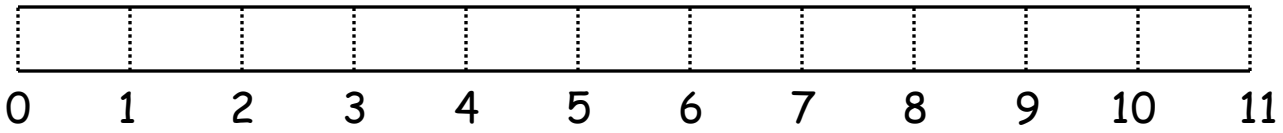
```
Sort jobs by finish times so that f_1 ≤ f_2 ≤ ... ≤ f_n.

jobs selected
A ← φ
for j = 1 to n {
   if (job j compatible with A)
      A ← A ∪ {j}
}
return A
```

Implementation. $O(n \log n)$.

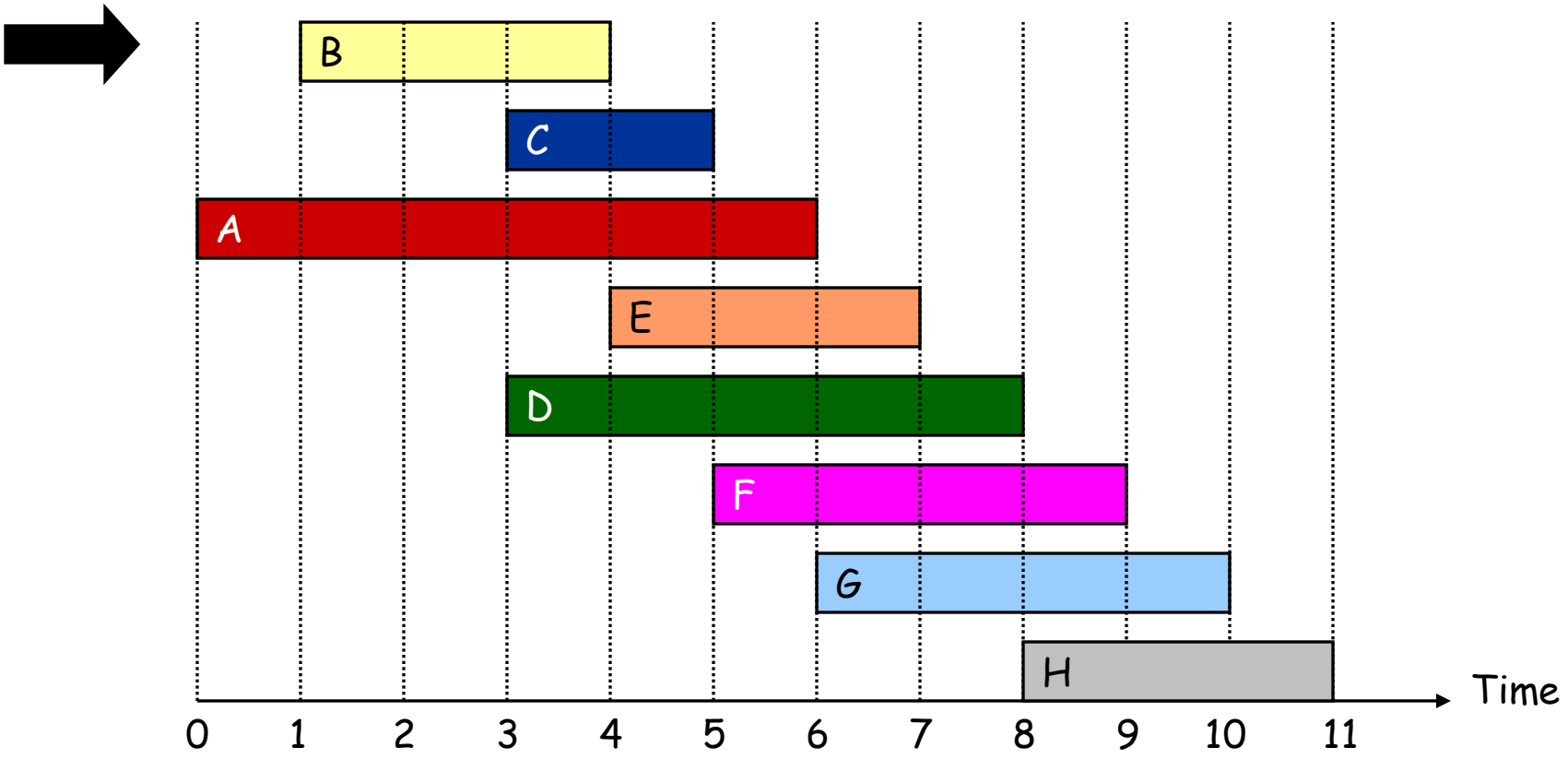
- Remember job j* that was added last to A.
- Job j is compatible with A iff $s_j \geq f_{j^*}$.

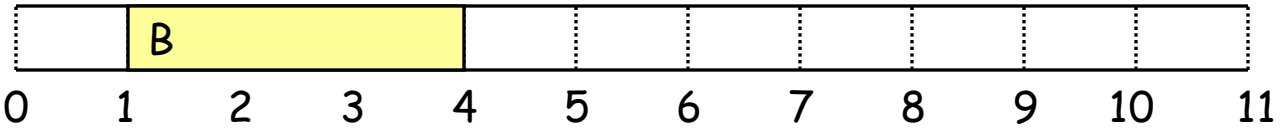
Interval Scheduling



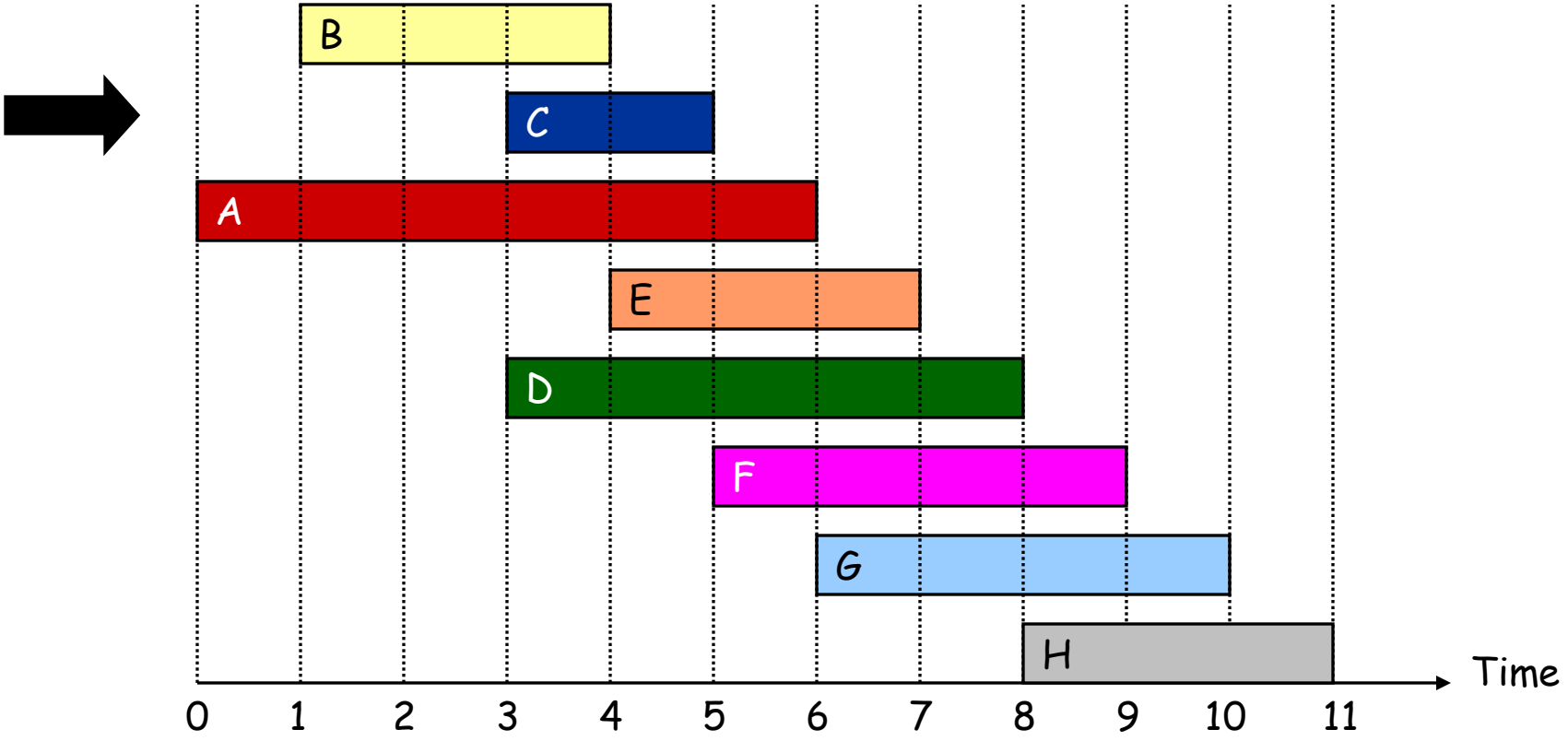


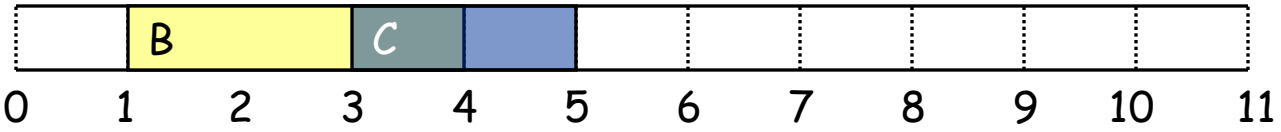
Interval Scheduling



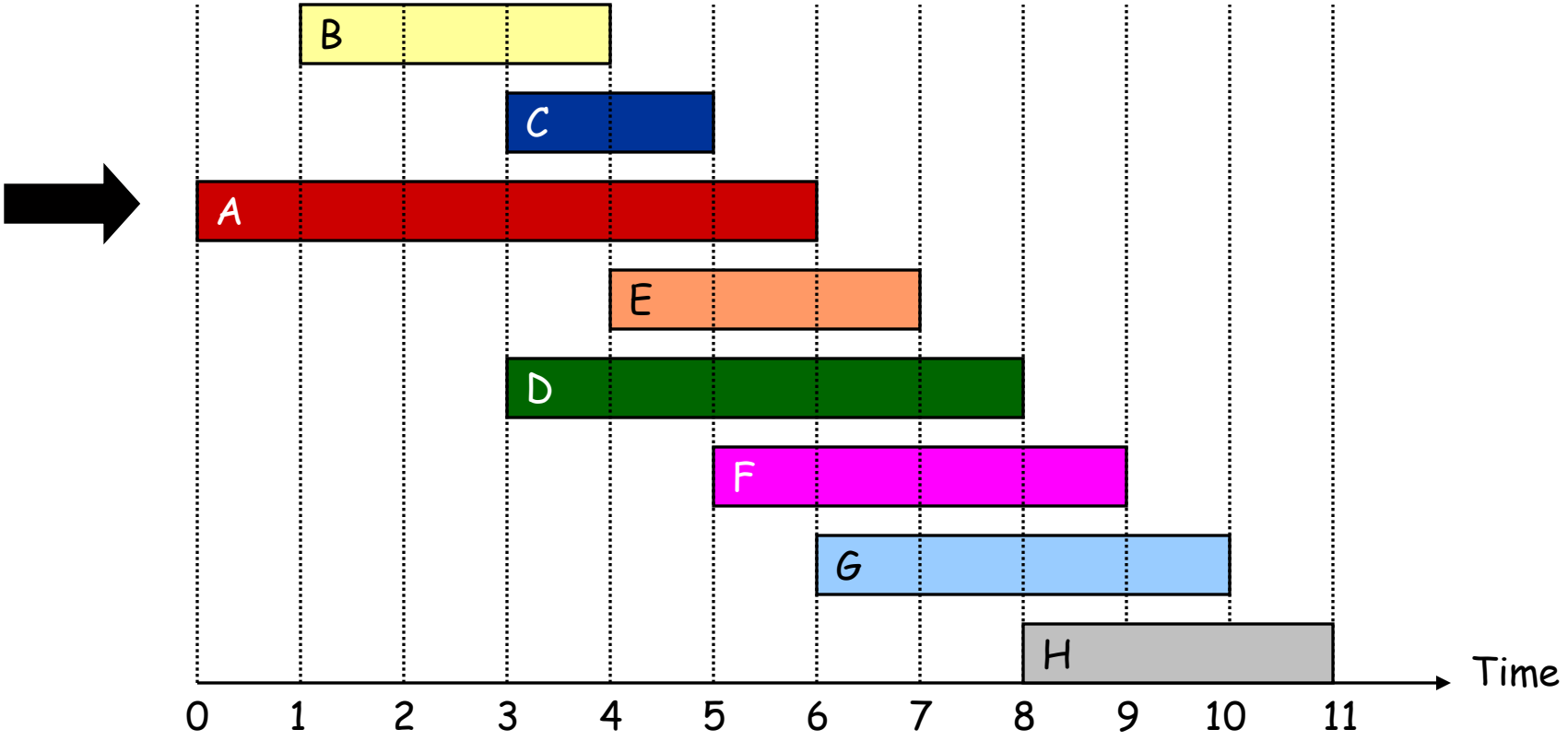


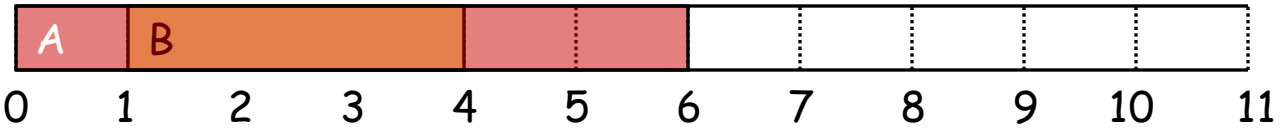
Interval Scheduling



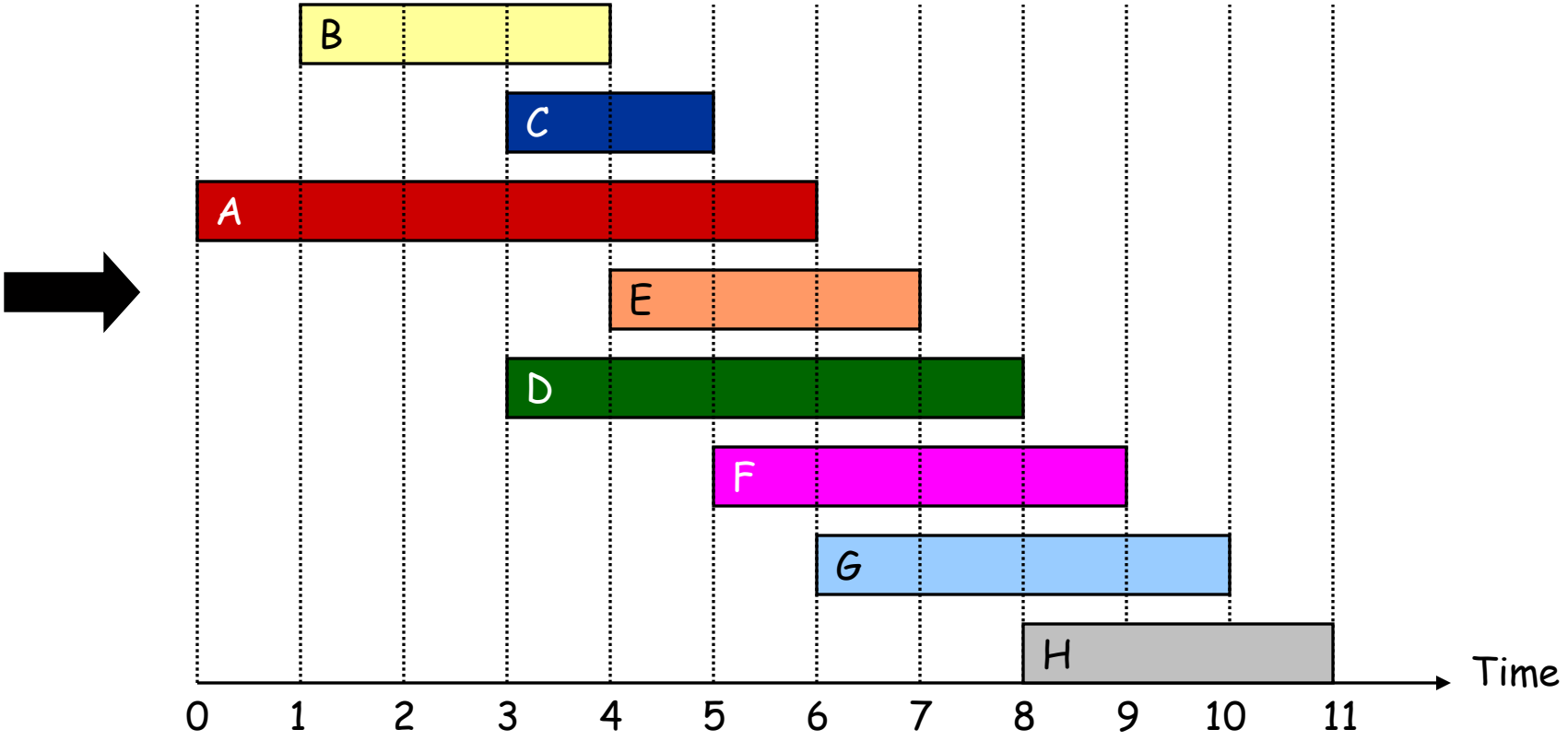


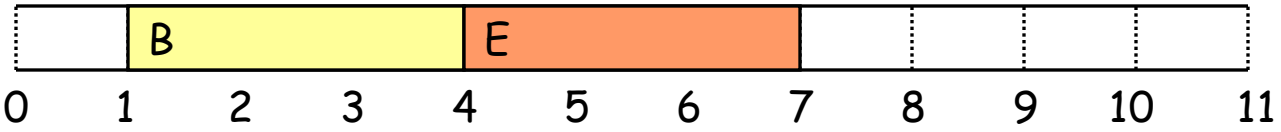
Interval Scheduling



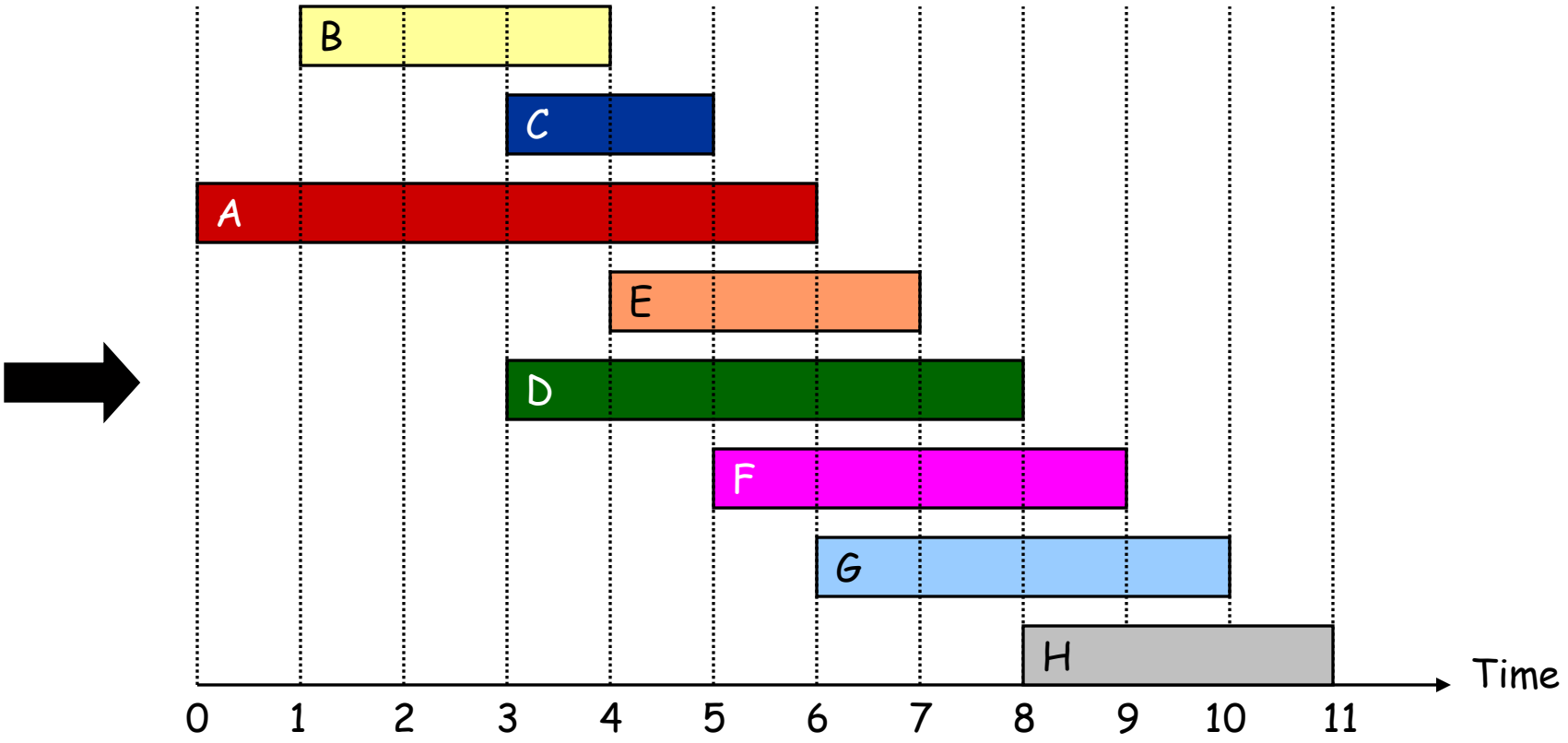


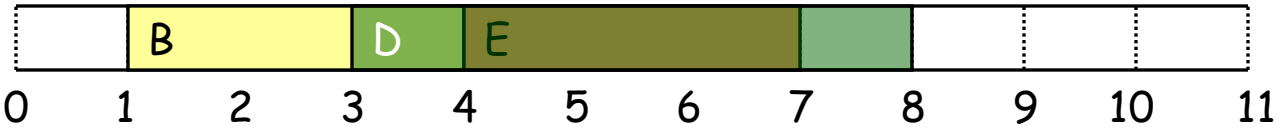
Interval Scheduling



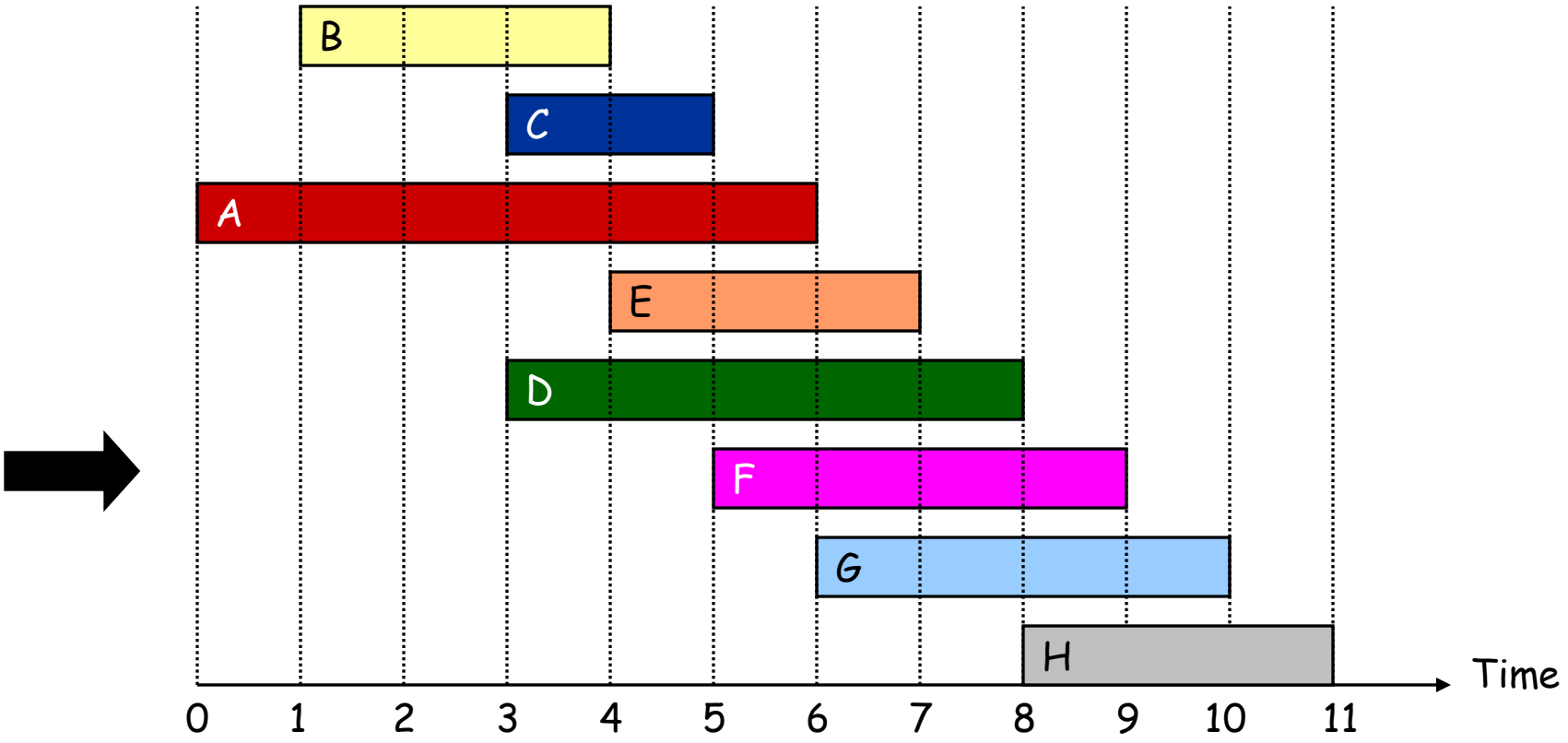


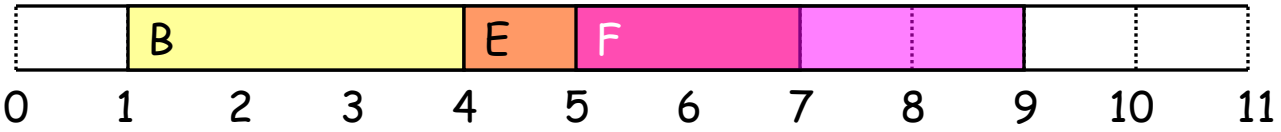
Interval Scheduling



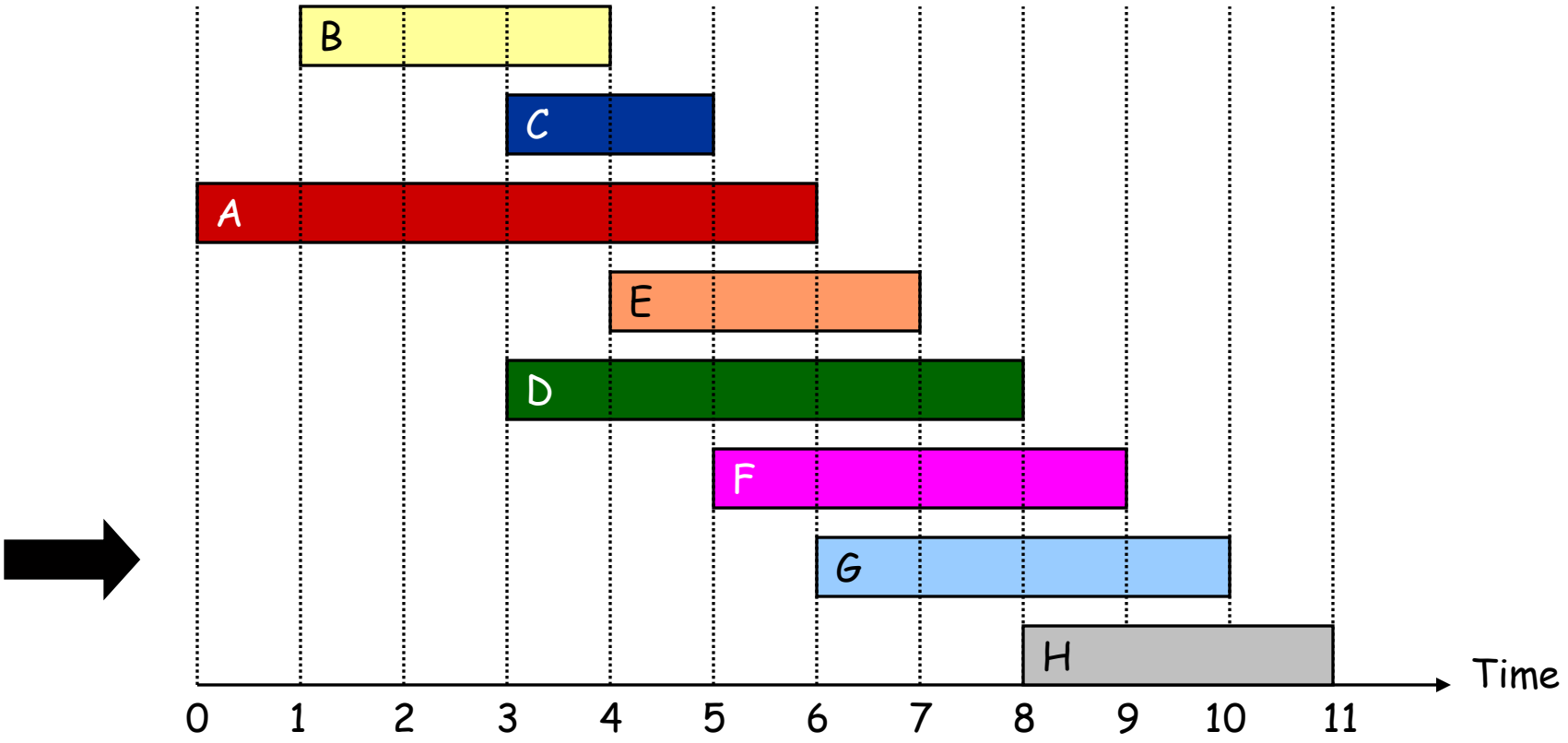


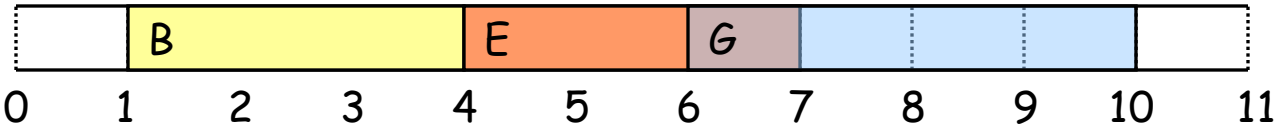
Interval Scheduling



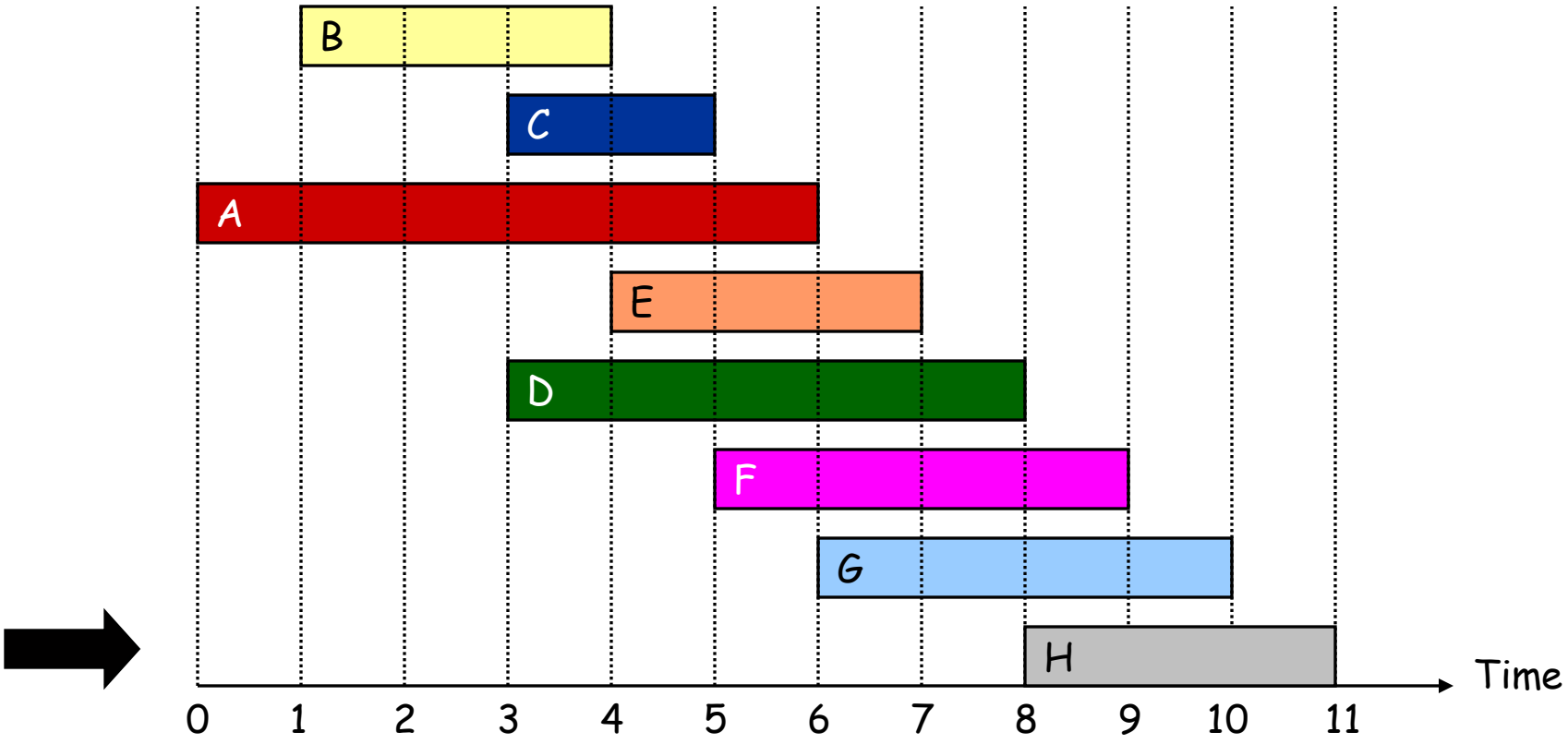


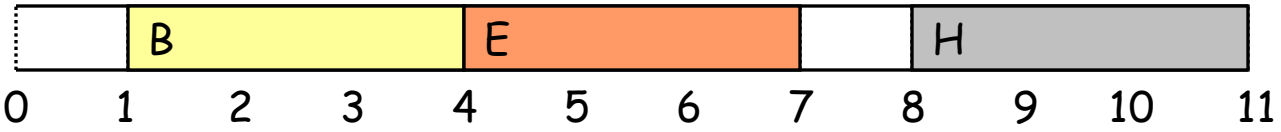
Interval Scheduling





Interval Scheduling



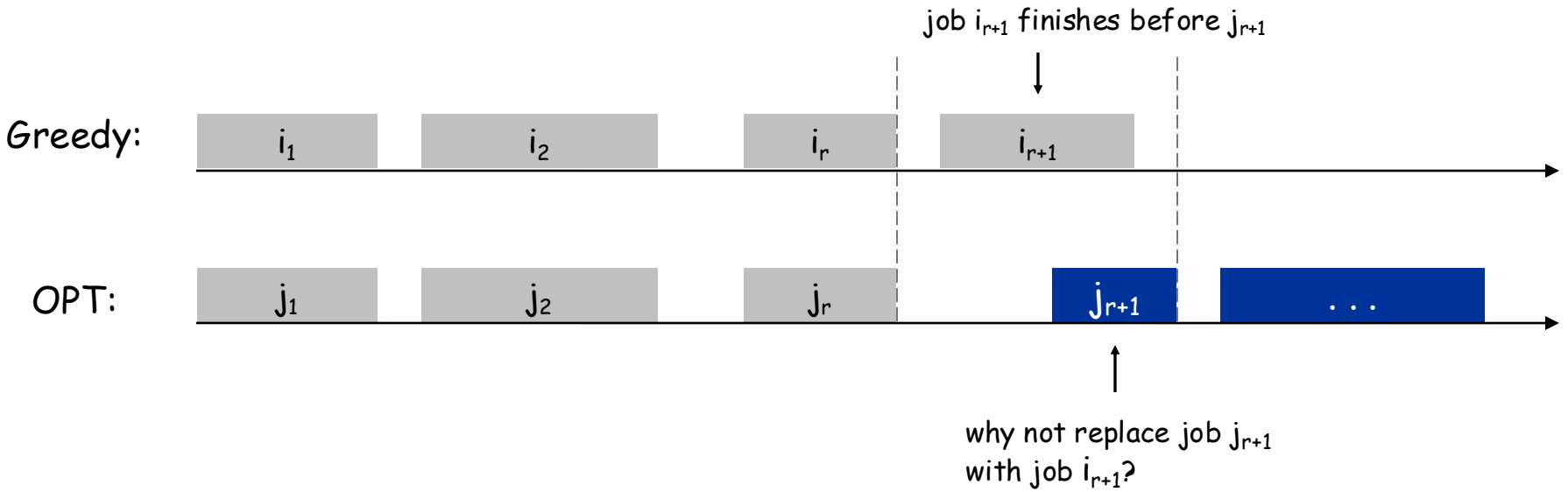


Interval Scheduling: Analysis

Theorem. Greedy algorithm is optimal.

Pf. (by contradiction)

- Assume greedy is not optimal, and let's see what happens.
- Let $i_1, i_2, \ldots i_k$ denote set of jobs selected by greedy.
- Let $j_1, j_2, \ldots j_m$ denote set of jobs in the optimal solution with $i_1 = j_1, i_2 = j_2, \ldots, i_r = j_r$ for the largest possible value of r.

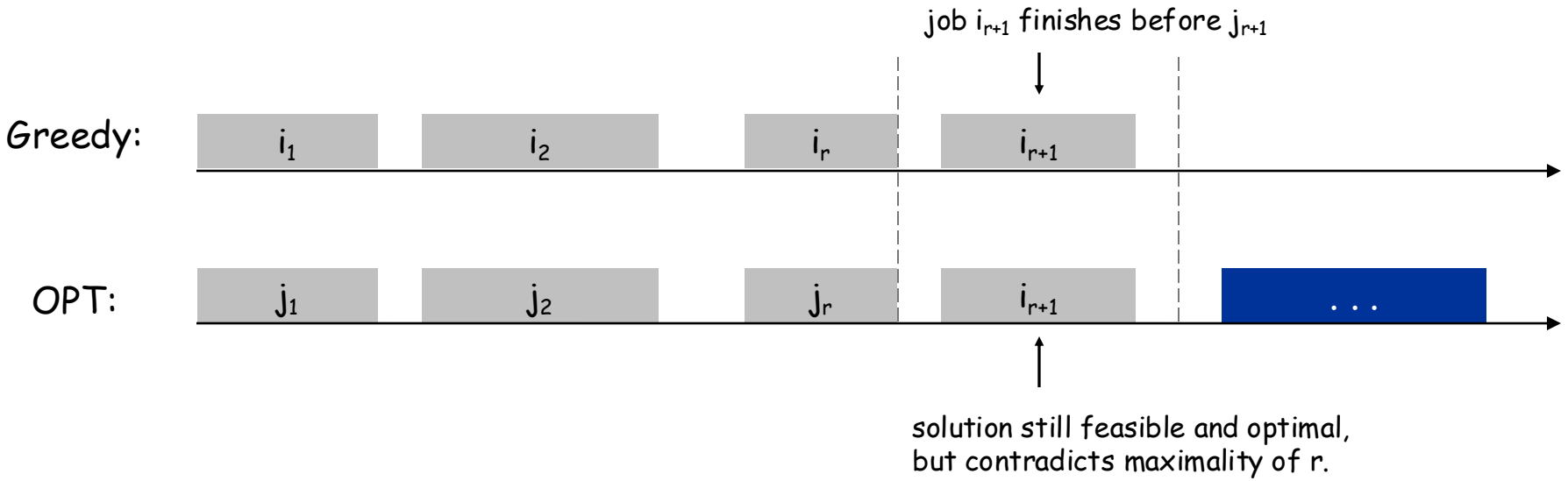


Interval Scheduling: Analysis

Theorem. Greedy algorithm is optimal.

Pf. (by contradiction)

- Assume greedy is not optimal, and let's see what happens.
- Let $i_1, i_2, \ldots i_k$ denote set of jobs selected by greedy.
- Let $j_1, j_2, \ldots j_m$ denote set of jobs in the optimal solution with $i_1 = j_1, i_2 = j_2, \ldots, i_r = j_r$ for the largest possible value of r.



Analysis

- If there are n intervals, sorting the intervals takes O(n log n) time.
- Then we go through the intervals in order of finishing times.
  - For each interval, check if it intersects last selected one, and select it if it doesn't.
  - Takes O(n) time.
- Total O(n log n) time.

4.2 Scheduling to Minimize Lateness

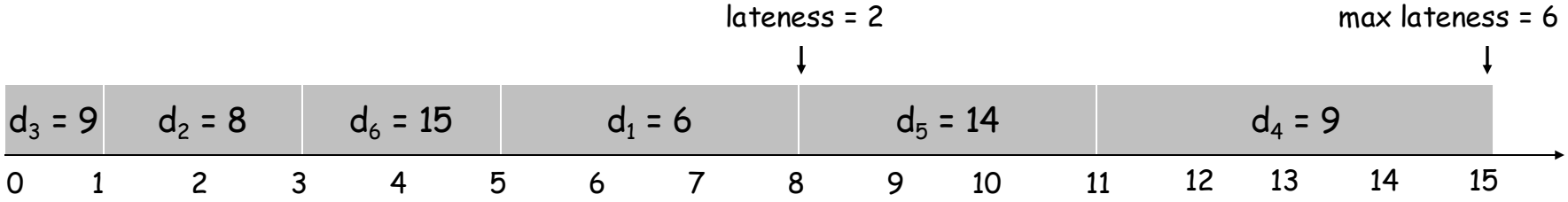
Scheduling to Minimizing Lateness

Minimizing lateness problem.

- Single resource processes one job at a time.
- Job j requires t_j units of processing time and is due at time d_j.
- If j starts at time s_j, it finishes at time $f_j = s_j + t_j$.
- Lateness: $\ell_j = \max \{ 0, f_j - d_j \}$.
- Goal: schedule all jobs to minimize maximum lateness $L = \max \ell_j$.

Ex:

	1	2	3	4	5	6
t_j	3	2	1	4	3	2
d_j	6	8	9	9	14	15



Minimizing Lateness: Greedy Algorithms

Greedy template. Consider jobs in some order.

- [Shortest processing time first] Consider jobs in ascending order of processing time t_j.

- [Earliest deadline first] Consider jobs in ascending order of deadline d_j.

- [Smallest slack] Consider jobs in ascending order of slack $d_j - t_j$.

Minimizing Lateness: Greedy Algorithms

Greedy template. Consider jobs in some order.

- [Shortest processing time first] Consider jobs in ascending order of processing time t_j.

	1	2
t_j	1	10
d_j	100	10

counterexample

- [Smallest slack] Consider jobs in ascending order of slack $d_j - t_j$.

	1	2
t_j	1	10
d_j	2	10

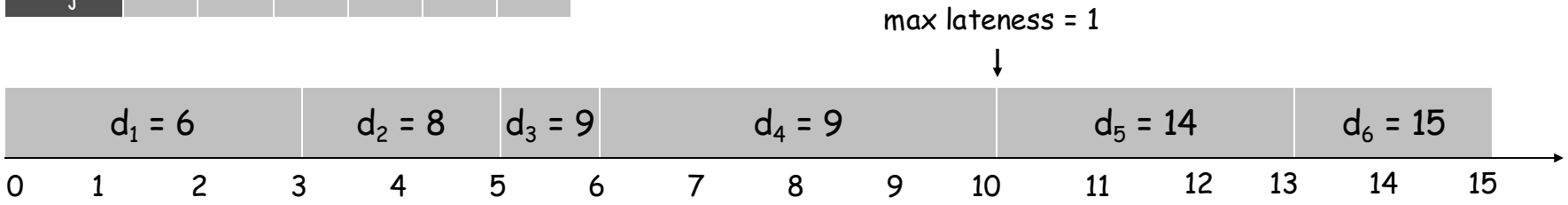
counterexample

Minimizing Lateness: Greedy Algorithm

Greedy algorithm. Earliest deadline first.

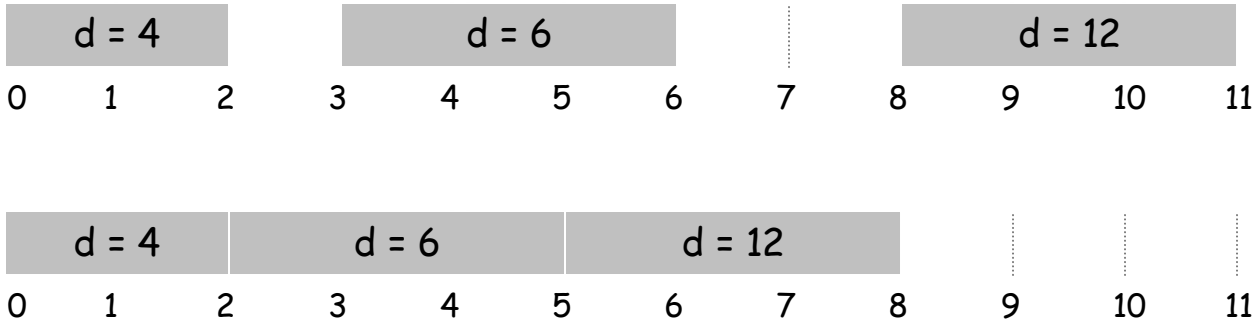
```
Sort n jobs by deadline so that d_1 ≤ d_2 ≤ … ≤ d_n

t ← 0
for j = 1 to n
   Assign job j to interval [t, t + t_j]
   s_j ← t, f_j ← t + t_j
   t ← t + t_j
output intervals [s_j, f_j]
```

	1	2	3	4	5	6
t_j	3	2	1	4	3	2
d_j	6	8	9	9	14	15



Minimizing Lateness: No Idle Time

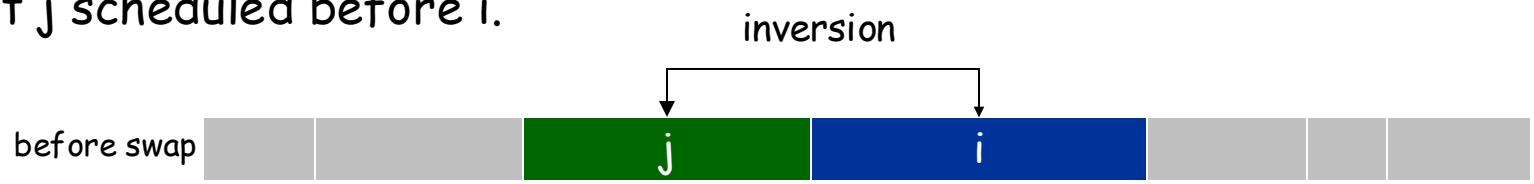
Observation. There exists an optimal schedule with no idle time.



Observation. The greedy schedule has no idle time.

Minimizing Lateness: Inversions

Def. An inversion in schedule S is a pair of jobs i and j such that: i < j but j scheduled before i.

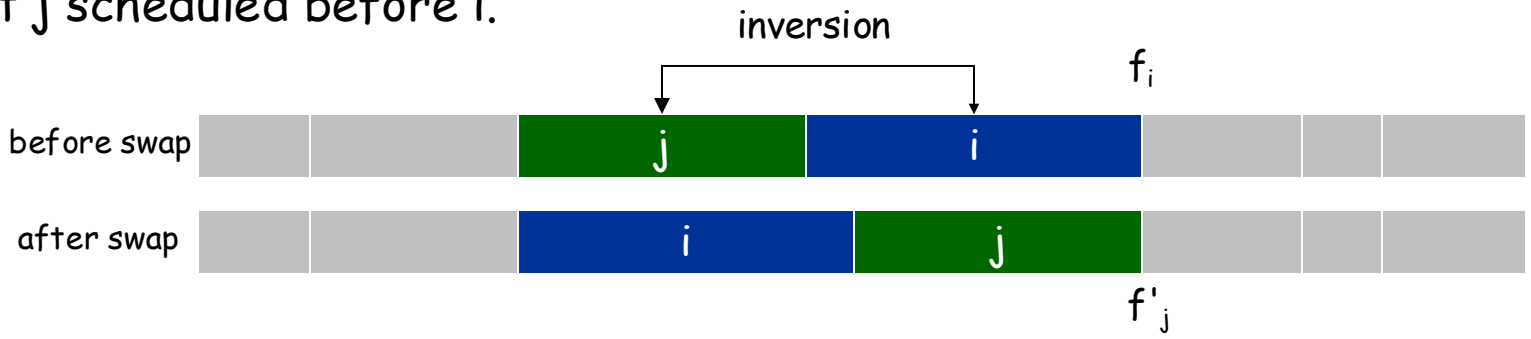


Observation. Greedy schedule has no inversions.

Observation. If a schedule (with no idle time) has an inversion, it has one with a pair of inverted jobs scheduled consecutively.

Minimizing Lateness: Inversions

Def. An inversion in schedule S is a pair of jobs i and j such that: i < j but j scheduled before i.



Claim. Swapping two adjacent, inverted jobs reduces the number of inversions by one and does not increase the max lateness.

Pf. Let ℓ be the lateness before the swap, and let ℓ' be it afterwards.

- $\ell'_k = \ell_k$ for all $k \neq i, j$
- $\ell'_i \leq \ell_i$
- If job j is late:

$$\begin{aligned} \ell'_j &= f_j' - d_j && \text{(definition)} \\ &= f_i - d_j && (j \text{ finishes at time } f_i) \\ &\leq f_i - d_i && (i < j) \\ &\leq \ell_i && \text{(definition)} \end{aligned}$$

Minimizing Lateness: Analysis of Greedy Algorithm

Theorem. Greedy schedule S is optimal.

Pf. Define S* to be an optimal schedule that has the fewest number of inversions, and let's see what happens.

- Can assume S* has no idle time.
- If S* has no inversions, then S = S*.
- If S* has an inversion, let i-j be an adjacent inversion.
  - swapping i and j does not increase the maximum lateness and strictly decreases the number of inversions
  - this contradicts definition of S* ▪

Greedy Analysis Strategies

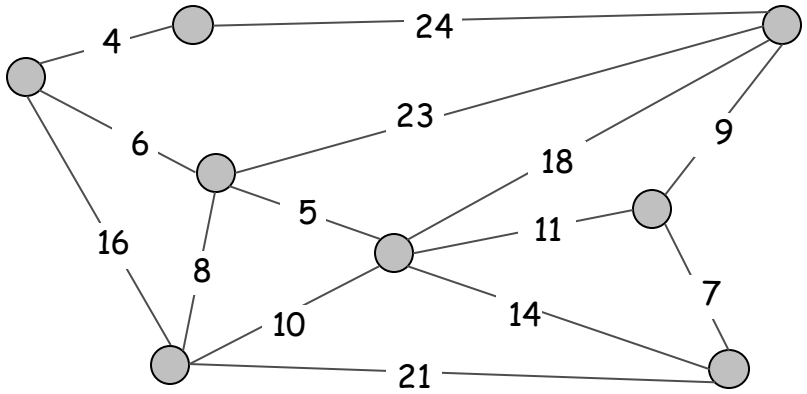
Greedy algorithm stays ahead. Show that after each step of the greedy algorithm, its solution is at least as good as any other algorithm's.

Exchange argument. Gradually transform any solution to the one found by the greedy algorithm without hurting its quality.

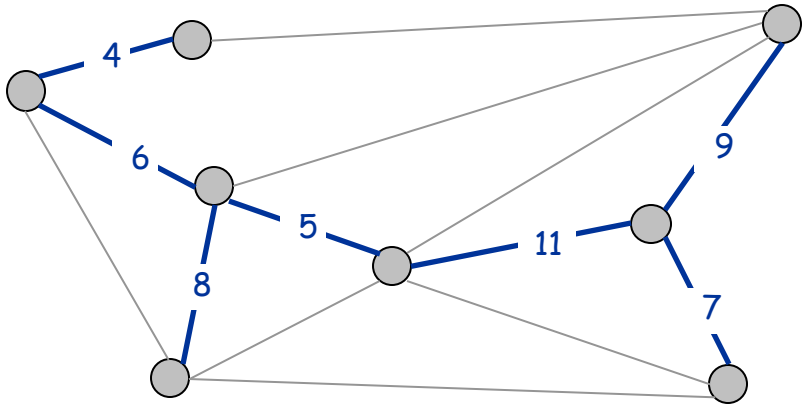
4.5 Minimum Spanning Tree

Minimum Spanning Tree

Minimum spanning tree. Given a connected graph $G = (V, E)$ with real-valued edge weights c_e, an MST is a subset of the edges $T \subseteq E$ such that T is a spanning tree whose sum of edge weights is minimized.



$G = (V, E)$



T, $\Sigma_{e \in T} c_e = 50$

Cayley's formula. There are n^{n-2} spanning trees of n vertices.

↑

can't solve by brute force

Applications

MST is fundamental problem with diverse applications.

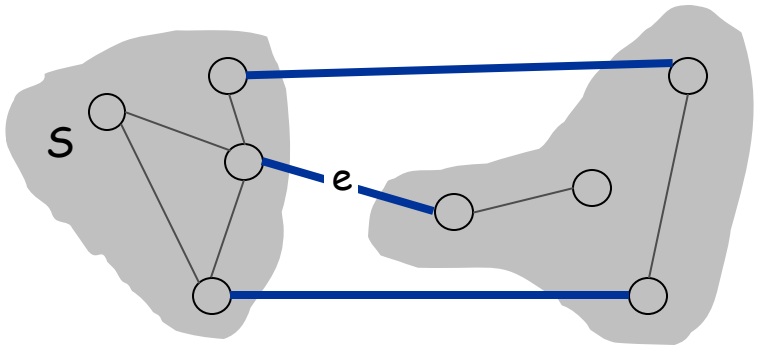
- Network design.
  - telephone, electrical, hydraulic, TV cable, computer, road
- Approximation algorithms for NP-hard problems.
  - traveling salesperson problem, Steiner tree
- Indirect applications.
  - max bottleneck paths
  - LDPC codes for error correction
  - image registration with Renyi entropy
  - learning salient features for real-time face verification
  - reducing data storage in sequencing amino acids in a protein
  - model locality of particle interactions in turbulent fluid flows
  - autoconfig protocol for Ethernet bridging to avoid cycles in a network
- Cluster analysis.

Greedy Algorithms

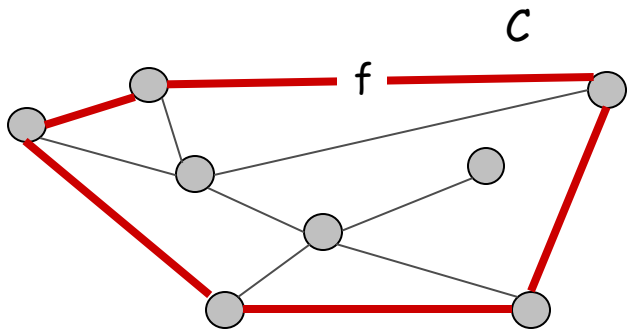
Simplifying assumption. All edge costs c_e are distinct.

Cut property. Let S be any subset of nodes, and let e be the min cost edge with exactly one endpoint in S. Then the MST contains e.

Cycle property. Let C be any cycle, and let f be the max cost edge belonging to C. Then the MST does not contain f.



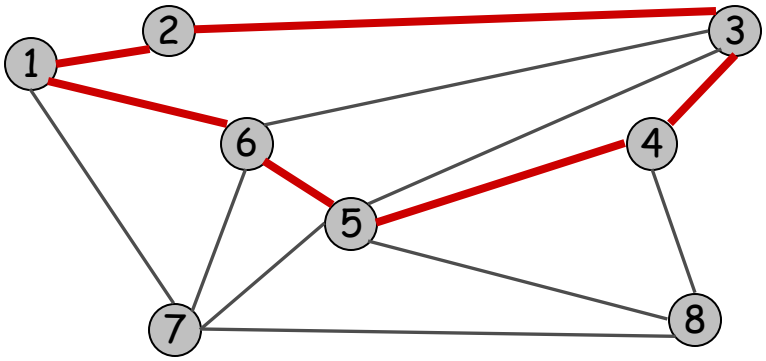
e is in the MST



f is not in the MST

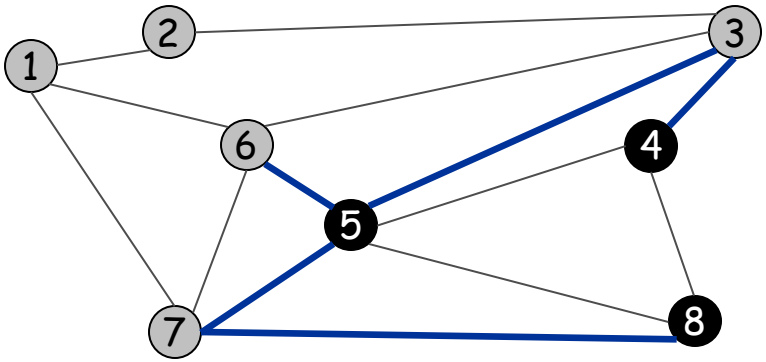
Cycles and Cuts

Cycle. Set of edges of the form a-b, b-c, c-d, …, y-z, z-a.



Cycle C = 1-2, 2-3, 3-4, 4-5, 5-6, 6-1

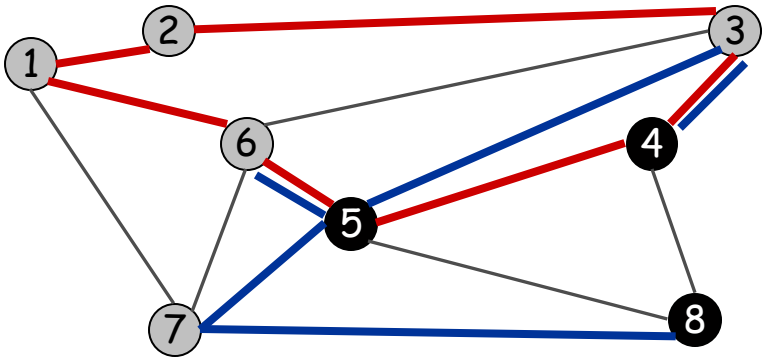
Cutset. A cut is a subset of nodes S. The corresponding cutset D is the subset of edges with exactly one endpoint in S.



Cut S = { 4, 5, 8 }
Cutset D = 5-6, 5-7, 3-4, 3-5, 7-8

Cycle-Cut Intersection

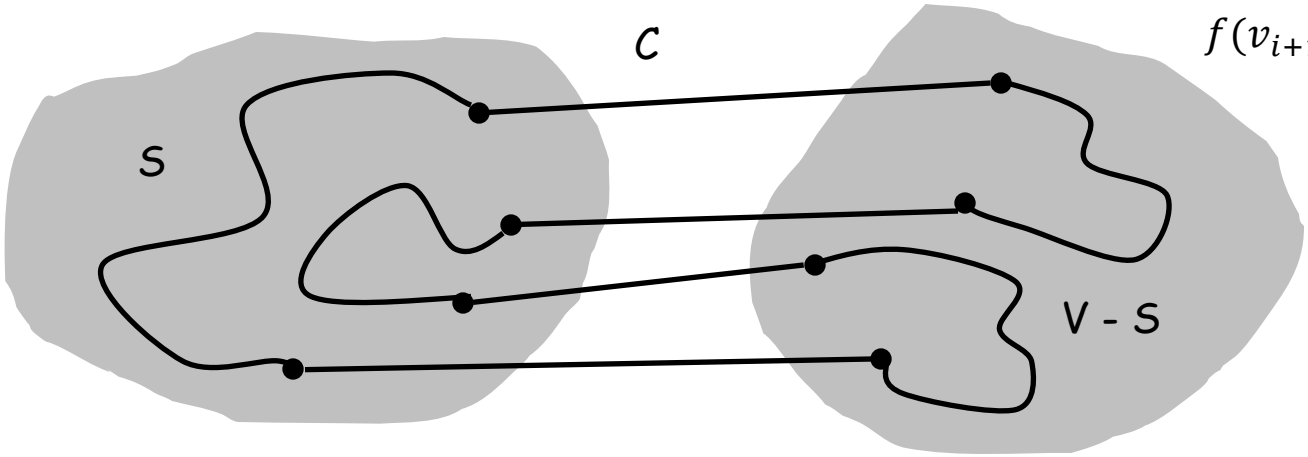
Claim. A cycle and a cutset intersect in an even number of edges.



Cycle C = 1-2, 2-3, 3-4, 4-5, 5-6, 6-1
Cutset D = 3-4, 3-5, 5-6, 5-7, 7-8
Intersection = 3-4, 5-6

Pf. (by picture)

- Indicator f(v) of whether node v from cycle C is in cut S.
- Common edge number: $\sum_{i=1}^{k}[f(v_i) \neq f(v_{i+1})]$



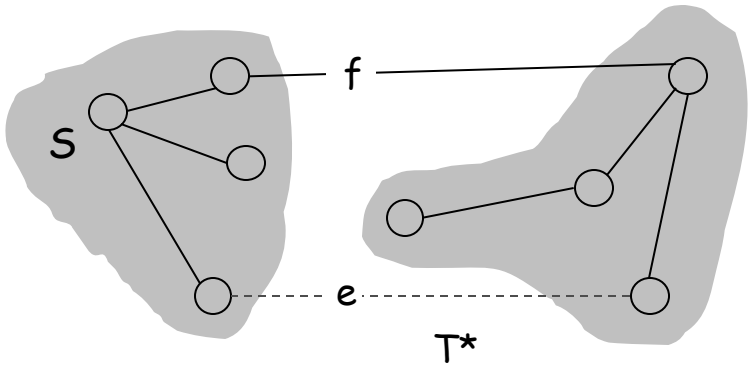
Greedy Algorithms

Simplifying assumption. All edge costs c_e are distinct.

Cut property. Let S be any subset of nodes, and let e be the min cost edge with exactly one endpoint in S. Then the MST T* contains e.

Pf. (exchange argument)

- Suppose e does not belong to T*, and let's see what happens.
- Adding e to T* creates a cycle C in T*.
- e is in a cycle C with exactly one endpoint in S $\Rightarrow$ there exists another edge f in C with exactly one endpoint in S.
- $T' = T^* \cup \{ e \} - \{ f \}$ is also a spanning tree.
- Since $c_e < c_f$, $cost(T') < cost(T^*)$.
- This is a contradiction. ▪



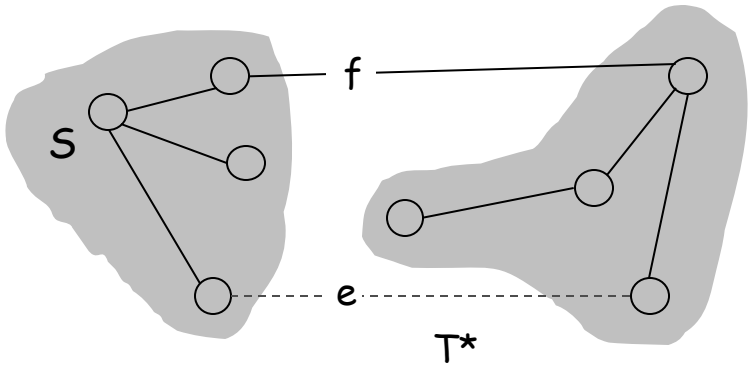
Greedy Algorithms

Simplifying assumption. All edge costs c_e are distinct.

Cycle property. Let C be any cycle in G, and let f be the max cost edge belonging to C. Then the MST T^* does not contain f.

Pf. (exchange argument)

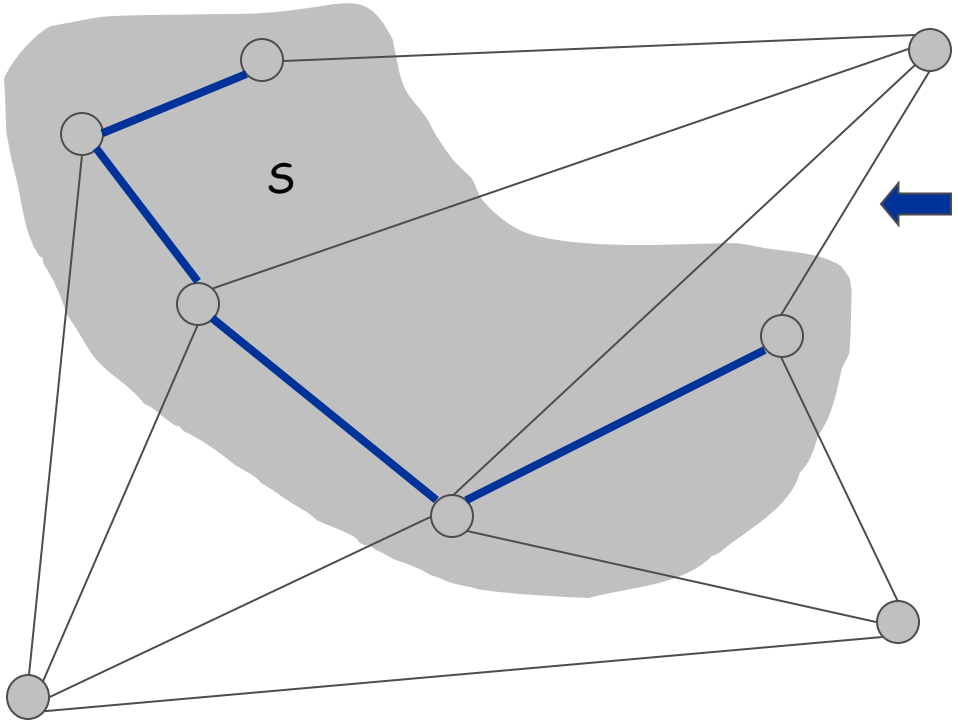
- Suppose f belongs to T^*, and let's see what happens.
- Deleting f from T^* creates a cut S in T^*.
- f is in the cycle C with exactly one endpoint in S $\Rightarrow$ there exists another edge e in C with exactly one endpoint in S.
- $T' = T^* \cup \{ e \} - \{ f \}$ is also a spanning tree.
- Since $c_e < c_f$, $cost(T') < cost(T^*)$.
- This is a contradiction. ▪



Prim's Algorithm: Proof of Correctness

Prim's algorithm. [Jarník 1930, Prim 1957, Dijkstra 1959]

- Initialize S = any node.
- Apply cut property to S.
- Add min cost edge in cutset corresponding to S to T, and add one new explored node u to S.



Implementation: Prim's Algorithm

Implementation. Use a priority queue.

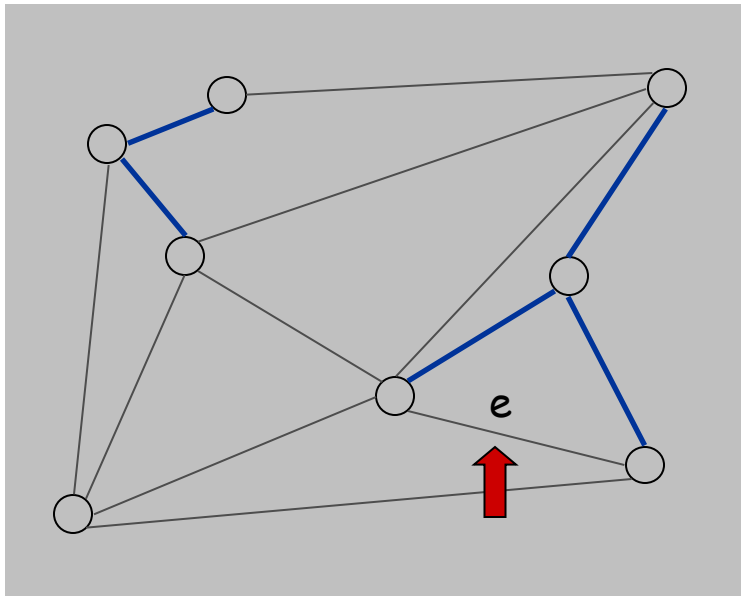
- Maintain set of explored nodes S.
- For each unexplored node v, maintain attachment cost a[v] = cost of cheapest edge from v to a node in S.
- $O(n^2)$ with an array; $O(m \log n)$ with a binary heap.

```
Prim(G, c) {
    foreach (v ∈ V) a[v] ← ∞
    Initialize an empty priority queue Q
    foreach (v ∈ V) insert v onto Q
    Initialize set of explored nodes S ← ϕ

    while (Q is not empty) {
        u ← delete min element from Q
        S ← S ∪ { u }
        foreach (edge e = (u, v) incident to u)
            if ((v ∉ S) and (c_e < a[v]))
                decrease priority a[v] to c_e
}
```

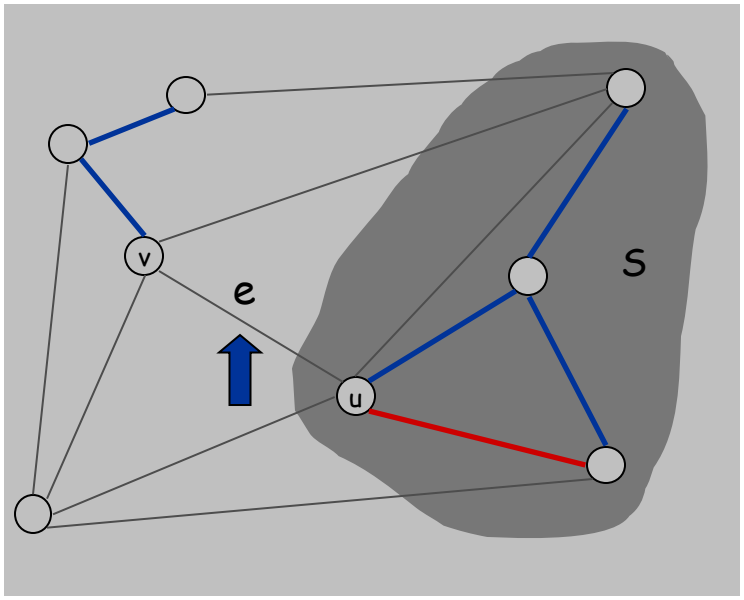
Kruskal's Algorithm: Proof of Correctness

Kruskal's algorithm. [Kruskal, 1956]

- Consider edges in ascending order of weight.
- Case 1: If adding e to T creates a cycle, discard e according to cycle property.
- Case 2: Otherwise, insert e = (u, v) into T according to cut property where S = set of nodes in u's connected component.
- Until (n-1) edges have been collected.

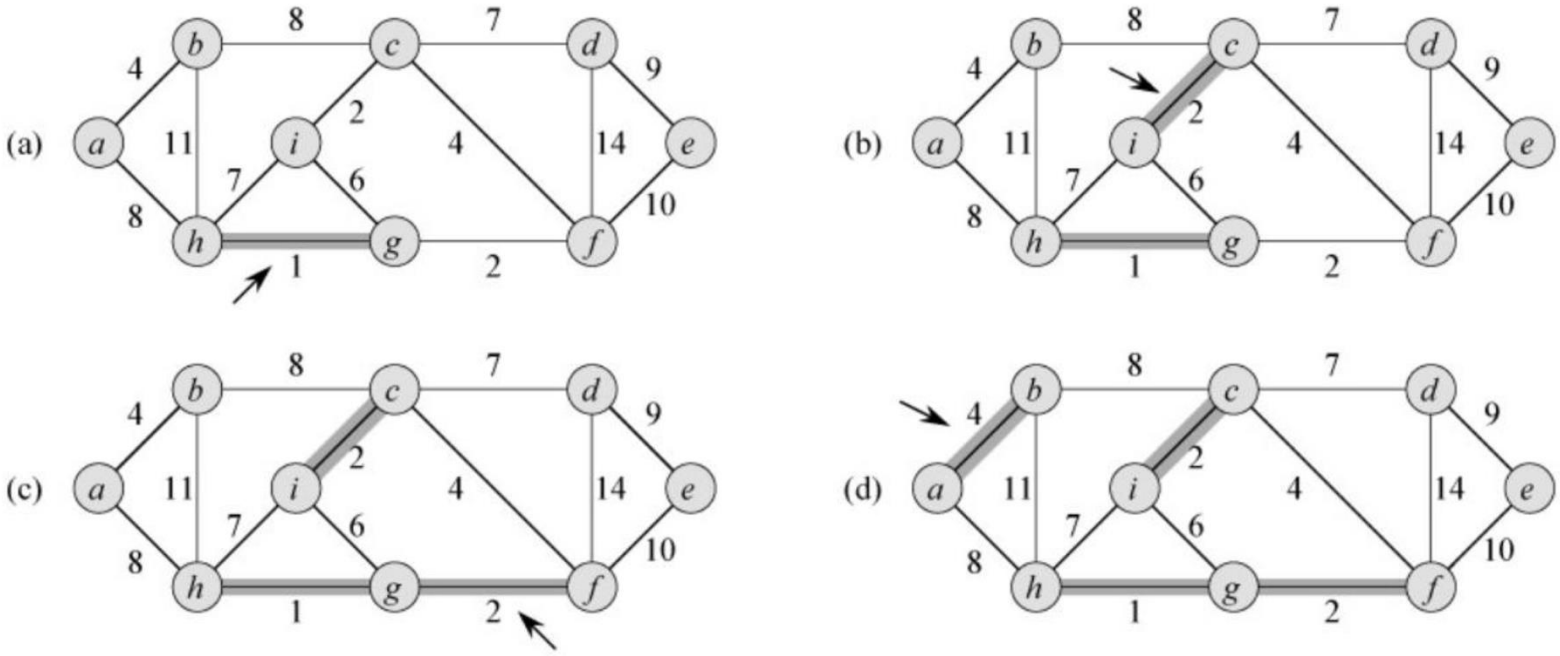


Case 1

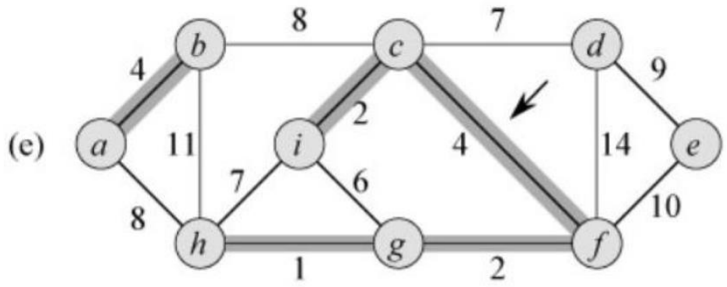


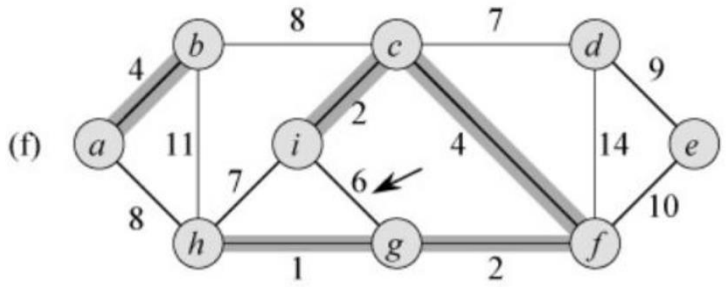
Case 2

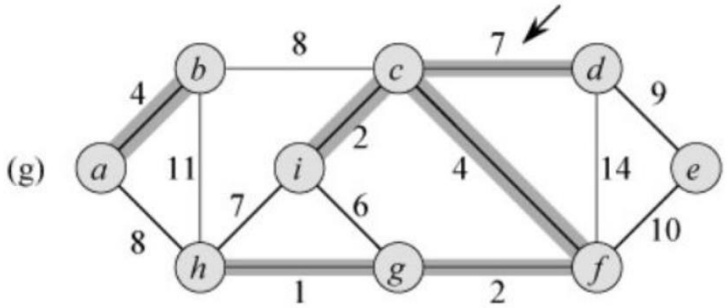
Kruskal's Algorithm: Proof of Correctness

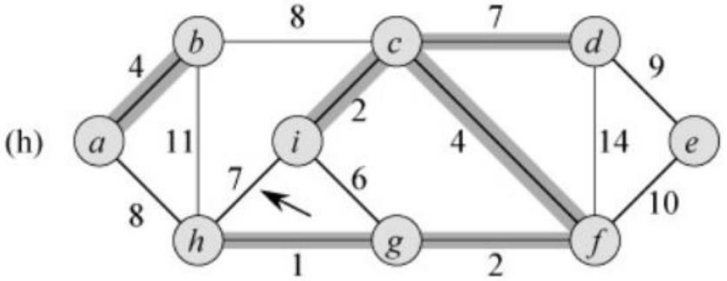


Kruskal's Algorithm: Proof of Correctness

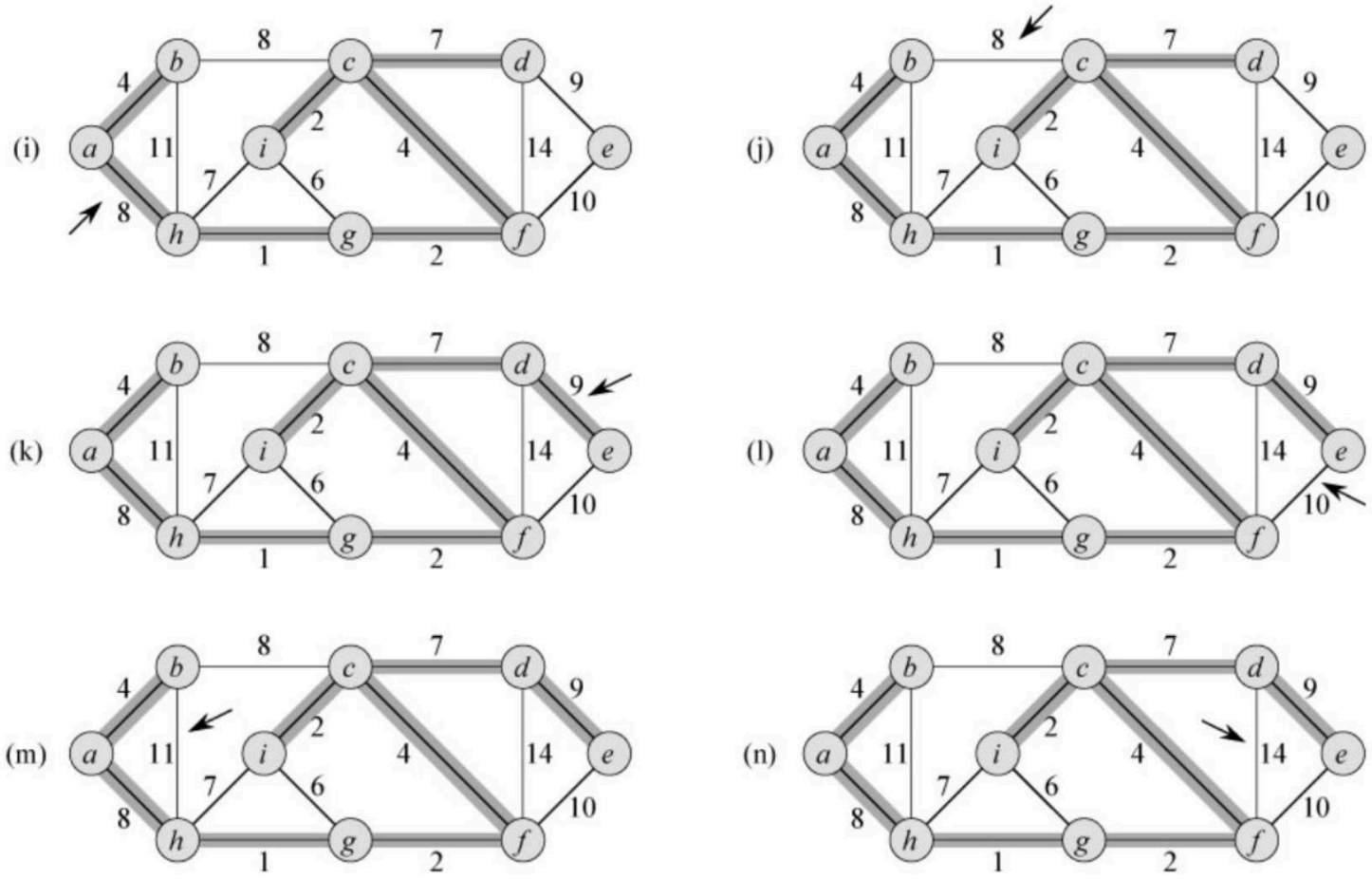








Kruskal's Algorithm: Proof of Correctness



Implementation: Kruskal's Algorithm

Implementation. Use the union-find data structure.

- Build set T of edges in the MST.
- Maintain set for each connected component.
- $O(m \log m)$ for sorting and $O(m \underbrace{\alpha(m, n)}_{\text{essentially a constant}})$ for union-find.

```
Kruskal(G, c) {
    Sort edges weights so that c1 ≤ c2 ≤ ... ≤ cm.
    T ← φ

    foreach (u ∈ V) make a set containing singleton u

    for i = 1 to m                  are u and v in different connected components?
        (u,v) = ei                  ↙
        if (u and v are in different sets) {
            T ← T ∪ {ei}
            merge the sets containing u and v
        }                           ↖ merge two components
    return T
}
```

Kruskal's Algorithm: Proof of Correctness

Three steps

1. Will prove that Kruskal's algorithm always produces a tree,
   - 1. Will prove that it produces an acyclic graph
   - 2. Will prove that it produces a connected graph

   => Kruskal's algorithm produces a tree T
2. Will prove that every edge chosen by Kruskal's algorithm is in every MST
3. => tree T produced by Kruskal's algorithm is a MST
   - This follows directly from (1) and (2).
   - From (1), it produces a tree T; T has |v| - 1 edges
   - From (2), every edge in T is in every MST
   - But, every MST contains exactly |v| - 1 edges so every MST must be T

Uses same simplifying assumption as in the proof of Prim's algorithm that all edges have different weights. Also assumes that input graph is connected; otherwise result could never be a tree

Kruskal's Algorithm: Proof of Correctness

Three steps

1. Will prove that Kruskal's algorithm always produces a tree,
   - a) prove that it produces an acyclic graph
   - b) prove that it produces a connected graph

   => Kruskal's algorithm produces a tree T

Proof:

Kruskal's algorithm starts with $T = \emptyset$ (the empty set) and only adds edge e if $T \cup \{e\}$ is acyclic

=> T remains acyclic through the entire algorithm

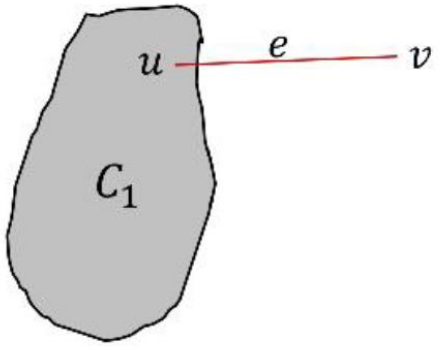
=> Final T output by the algorithm is acyclic

Kruskal's Algorithm: Proof of Correctness

Three steps

1. Will prove that Kruskal's algorithm always produces a tree,
   - a) prove that it produces an acyclic graph
   - b) prove that it produces a connected graph

   => Kruskal's algorithm produces a tree T



Proof:

Suppose T is not connected.

Let C1 be a connected component of T

=> There must exist some u ∈ C1 with e = (u, v) in G but v ∉ C1, because otherwise G is not connected

But adding e to T would not cause a cycle with any edges in T

=> Kruskal's algorithm would have added e to T, so e ∈ T

=> v ∈ C1

=> Contradiction => T is connected

Kruskal's Algorithm: Proof of Correctness

Three steps

2. Will prove that every edge chosen by Kruskal's algorithm is in every MST
   - Given: Cut lemma. Let S be any subset of nodes, and let e be the min cost edge with exactly one endpoint in S. Then every MST must contain e.

Proof of (2): Let e = (u, v) be in T.

- Plan: Will display S s.t. e is min-cost edge with exactly one endpoint in S.

Kruskal's Algorithm: Proof of Correctness

Three steps

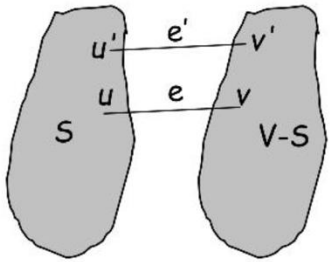
2. Will prove that every edge chosen by Kruskal's algorithm is in every MST

a) Consider the set of edges in T right before e was added

Let S = C_u be the vertices in the connected component containing u at that time

b) Since e was added to T, it doesn't create a cycle, so v ∉ S.

=> e has exactly one endpoint in S.



c) Suppose there exists e' = (u', v') with w(e') < w(e) s.t.

e' also has exactly one endpoint in S, i.e., u' ∈ S, v' ∉ S.

e' would have been processed by Kruskal's algorithm before e.

But e' would not have caused a cycle, so e' would have been added to T.

But then u', v' would be in the SAME component at the time e was being processed

so v' ∈ S, contradicting v' ∉ S.

Kruskal's Algorithm: Proof of Correctness

Three steps completed

1. Prove that Kruskal's algorithm always produces a tree,
   - a) proved that it produces an acyclic graph
   - b) proved that it produces a connected graph

   => Kruskal's algorithm produces a tree T
2. Proved that every edge chosen by Kruskal's algorithm is in every MST
3. => tree T produced by Kruskal's algorithm is a MST

This follows directly from (1) and (2).

- From (1), it produces a tree T; T has |v| - 1 edges
- From (2), every edge in T is in every MST

But, every MST contains exactly |v| - 1 edges so every MST must be T

Lexicographic Tiebreaking

To remove the assumption that all edge costs are distinct: perturb all edge costs by tiny amounts to break any ties.

Impact. Kruskal and Prim only interact with costs via pairwise comparisons. If perturbations are sufficiently small, MST with perturbed costs is MST with original costs.

e.g., if all edge costs are integers, perturbing cost of edge e_i by i / n^2

Implementation. Can handle arbitrarily small perturbations implicitly by breaking ties lexicographically, according to index.

```
boolean less(i, j) {
   if      (cost(e_i) < cost(e_j)) return true
   else if (cost(e_i) > cost(e_j)) return false
   else if (i < j)                 return true
   else                            return false
}
```

Minimum Spanning Tree

Prim's algorithm. Start with some root node s and greedily grow a tree T from s outward. At each step, add the cheapest edge e to T that has exactly one endpoint in T.

Kruskal's algorithm. Start with $T = \phi$. Consider edges in ascending order of cost. Insert edge e in T unless doing so would create a cycle.

Reverse-Delete algorithm. Start with T = E. Consider edges in descending order of cost. Delete edge e from T unless doing so would disconnect T.

Remark. All three algorithms produce an MST.